Python Classes and Methods COMS 261

1. Suppose we execute the following code.

```
1  class Counter:
2      def __init__(self):
3          self.count = 0
4
5      def increment(self):
6          self.count += 1
7
8  first = Counter()
9  second = Counter()
10 first.increment()
11 first.increment()
12 second.increment()
```

   (a) What is the value of `first.count`?

   (b) What is the value of `second.count`?

2. Decide whether each method below should be an instance method or a static method. Also decide if it should mutate the object in place or return a new object.

   (a) A method for a `BankAccount` object that deposits money into an account.

   (b) A method that converts miles to km and does not use any information stored in an object.

   (c) A method for a `Student` object that returns the student's current grade.

3. The following class contains an error. Where is it?

```
1  class Book:
2      def __init__(self, title):
3          self.title = title
4
5      def describe():
6          print("This book is called", self.title)
7
8  my_book = Book("The Hobbit")
9  my_book.describe()
```

4. Consider the following class.

```
class MathTools:
    def double(number):
        return 2 * number

    def triple(self, number):
        return 3 * number
```

   (a) How would you call the `double` method if you wanted to double the number 5?

   (b) You can create an instance of `MathTools` named `tools` by entering `tools = MathTools()` even though there is no constructor function. How would you use `tools` to call `triple` to calculate three times 7?

   (c) What error do you get if you try this command: `MathTools.triple(7)`?

5. Write a class named `Rectangle` with the following features:

   - The constructor takes a width and a height and stores them as instance attributes.
   - An instance method named `perimeter` returns the rectangle's perimeter.
   - A static method named `new_square` inputs a single number and returns a new Rectangle object with that number as its width and height.

6. Create two classes: one called `Song` that has attributes `artist`, `title`, and `length`; the other called `Playlist` that has a single attribute called `songs` that is a list of Song objects. Then write a method called `total_duration` that finds the total duration of all the songs in the playlist. You can assume that each song's length is measured in seconds. Here is an example of how the method should be called:

```
my_songs = Playlist([
    Song("Help!", "The Beatles", 138),
    Song("I Need Some of That", "Weezer", 199)
    Song("Money", "Pink Floyd", 239)
])
print(my_songs.total_duration()) # Should print 576
```